VECTOR

MICROSAR Wdg

Technical Reference

Sbc watchdog driver

Version 2.00.00

Authors	Matthias Scheid
Status	Released

Document Information

History

Author	Date	Version	Remarks
Matthias Scheid	2016-02-09	2.00.00	Creation of document

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_WatchdogDriver.pdf	V2.5.0
[2]	AUTOSAR	AUTOSAR_BasicSoftwareModules.pdf	V1.0.0
[3]	Vector	Technical_Reference_ Wdg_general.pdf	V3.0.0
[4]	Vector	Technical_Reference_Sbc_<derivate>.pdf	-

Scope of the Document

This technical reference describes the specific use of the Wdg_30_Sbc driver software. It supplements the general Wdg driver technical reference [3].



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

© 2016 Vector Informatik GmbH

Contents

1 Component History 5

2 Introduction 6
2.1 Architecture Overview 6

3 Functional Description 7
3.1 Features 7
3.2 Initialization 7

4 Integration 8
4.1 Scope of Delivery 8
4.2 Interaction between watchdog module and Sbc module 8
4.2.1 Initial trigger 8
4.2.2 Hardware specific mode change behavior 9
4.2.3 Modes supported by hardware 9
4.3 Trigger timing 11
4.3.1 Gpt Channel 11
4.3.2 Triggering time 11

5 API Description 13
5.1 Type Definitions 13
5.2 Services used by WDG 14

6 Glossary and Abbreviations 15
6.1 Abbreviations 15

7 Contact 16

Illustrations

Figure 2-1 AUTOSAR 4.x Architecture Overview 6
Figure 4-1 Configuration of SbcWatchdogPeripheral in DaVinci Configurator 11

Tables

Table 1-1 Component history 5
Table 4-1 Macro WDG_30_SBC_INITIAL_TRIGGER 8
Table 4-2 Supported types of mode switches 9
Table 4-3 Macro WDG_30_SBC_MODE_CHANGE_ASYNC 9
Table 4-4 Macro WDG_30_SBC_SLOW_MODE_ALLOWED 10
Table 4-5 Macro WDG_30_SBC_OFF_MODE_ALLOWED 10
Table 5-1 Type Defenitions 13
Table 5-2 Wdg_ConfigType 14
Table 5-3 Services used by the WDG 14
Table 6-1 Abbreviations 15

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.00	Creation of Wdg_30_Sbc
2.00.00	Updated component according to SafeBSW process (R14)

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module WDG as specified in [1].

Supported AUTOSAR Release*:	4.0.3	
Supported Configuration Variants:	**pre-compile**	
Vendor ID:	WDG_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	WDG_MODULE_ID	102 decimal (according to ref. [2])

* For the detailed functional specification please also refer to the corresponding AUTOSAR SWS.

The module provides services for initializing the window watchdog of Infineon TLF35584. Furthermore it supports changing watchdog timing configuration and triggering of the watchdog.

2.1 Architecture Overview

The following figure shows where the WDG is located in the AUTOSAR architecture.

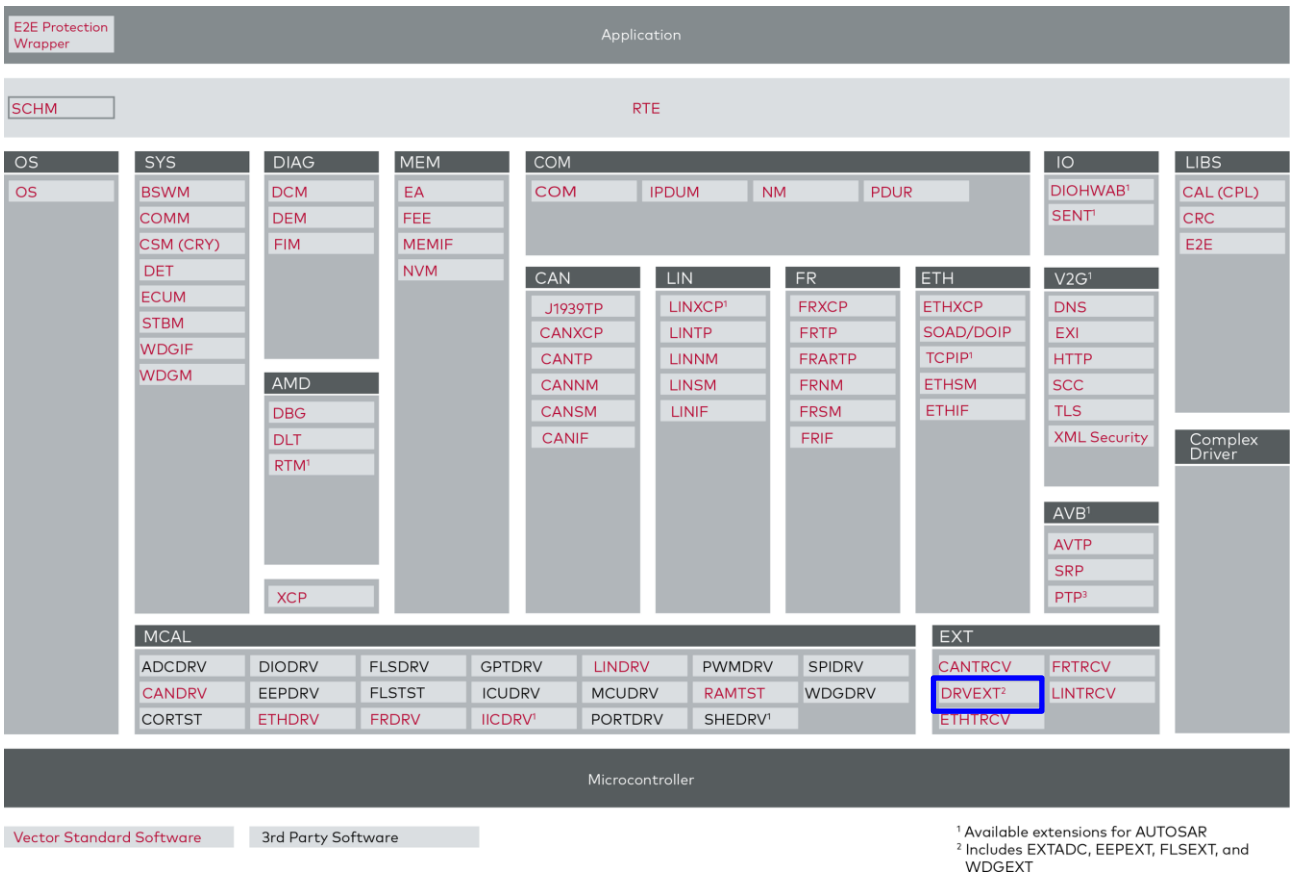


Figure 2-1 AUTOSAR 4.x Architecture Overview

3 Functional Description

3.1 Features

The module supports the features specified in Autosar (see [1]) as described in [3].

3.2 Initialization

The watchdog driver module is initialized as described in [3].

4 Integration

This chapter gives necessary information for the integration of the MICROSAR WDG into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the WDG contains the files which are described in [3].

An infix to all file names, APIs, memory sections, etc. is applied. (E.g. `Wdg.c` is infixed to `Wdg_30_Sbc.c`, `Wdg_Init()` is infixed to `Wdg_30_Sbc_Init()`).

4.2 Interaction between watchdog module and Sbc module

The watchdog driver controls the watchdog of a referenced Sbc module. The reference has to be set in parameter `WdgSbcRef` in DaVinci Configurator 5.

As Sbc devices have hardware-specific characteristics which have to be considered by the watchdog module, the Sbc module coordinates the watchdog driver behavior with a set of defines (in Sbc_30_<derivate>_Cfg.h) which describes this hardware-specific watchdog characteristics.

The hardware specific configuration macros of the Sbc module are mapped to watchdog configuration macros in `Wdg_30_Sbc_Cfg.h`

4.2.1 Initial trigger

As a referenced SBC may require a trigger to leave it´s INIT state the watchdog component can be configured by the Sbc to trigger the watchdog hardware during the initialization routine. To control the initial trigger the Sbc defines the preprocessor macro `SBC_30_<derivate>_WDG_INITIAL_TRIGGER` which is mapped to the watchdog configuration macro `WDG_30_SBC_INITIAL_TRIGGER`. Table 4-1 describes the influence of these macros to the behavior of the watchdog module.

Macro name	`WDG_30_SBC_INITIAL_TRIGGER (STD_ON /STD_OFF)`
Mapped from	`SBC_30_<derivate>_WDG_INITIAL_TRIGGER (STD_ON/STD_OFF)`
Settings:	`STD_ON`: The watchdog module calls the trigger function of the referenced Sbc during initialization
	`STD_OFF`: The watchdog module does not call the trigger function of the referenced Sbc.

Table 4-1 Macro WDG_30_SBC_INITIAL_TRIGGER

4.2.2 Hardware specific mode change behavior

Depending on the hardware characteristics of the watchdog included in the SBC, the mode change behavior can be adapted by the referenced Sbc module. The supported types of mode changes are described in Table 4-2.

To adapt the mode change behavior, the Sbc module defines the preprocessor macro `SBC_30_<derivate>_WDG_ASYNC_MODE_CHANGE` which is mapped to `WDG_30_SBC_MODE_CHANGE_ASYNC`. Table 4-3 describes the influence of these macros to the behavior of the watchdog module.

Mode change	Description
Asynchronous	A requested mode change is executed after the next watchdog trigger. This means that the trigger timing (managed by the watchdog module) has to be adapted only after the next trigger. If the SBC hardware requires this type of mode change it provides the preprocessor switch `SBC_30_<derivate>_WDG_ASYNC_MODE_CHANGE` with the value `STD_ON`.
Synchronous	A requested mode change is executed immediately by the watchdog hardware. This means that trigger timing managed by the watchdog module also has to be adapted immediately. If the SBC hardware requires this type of mode change it provides the preprocessor switch `SBC_30_<derivate>_WDG_ASYNC_MODE_CHANGE` with the value `STD_OFF`.

Table 4-2 Supported types of mode switches

Macro name	`WDG_30_SBC_MODE_CHANGE_ASYNC (STD_ON /STD_OFF)`
Mapped from	`SBC_30_<derivate>_WDG_MODE_CHANGE_ASYNC (STD_ON/STD_OFF)`
Settings:	`STD_ON`: The watchdog module performs an asynchronous mode change.
	`STD_OFF`: The watchdog module performs a synchronous mode change.

Table 4-3 Macro WDG_30_SBC_MODE_CHANGE_ASYNC

4.2.3 Modes supported by hardware

As some Sbc devices do not support disabling the watchdog or only provide a fixed timing. `WDGIF_SLOW_MODE` and/or `WDGIF_OFF_MODE` may not be supported. To adapt the behavior of the watchdog module properly the Sbc module defines the preprocessor macros

- `SBC_30_<derivate>_WDG_SLOW_MODE_ALLOWED` which is mapped to `WDG_30_SBC_SLOW_MODE_ALLOWED`

> SBC_30_<derivate>_WDG_OFF_MODE_ALLOWED which is mapped to WDG_30_SBC_OFF_MODE_ALLOWED

Table 4-4 and Table 4-5 describe the influence of these macros to the behavior of the watchdog module.

Note that WDGIF_FAST_MODE is always supported.

Macro name	WDG_30_SBC_SLOW_MODE_ALLOWED (STD_ON /STD_OFF)
Mapped from	SBC_30_<derivate>_WDG_SLOW_MODE_ALLOWED (STD_ON/STD_OFF)
Settings:	STD_ON: Mode WDGIF_SLOW_MODE is supported and can be used as Default Mode in the configuration structure passed to Wdg_30_Sbc_Init() or requested with Wdg_30_Sbc_SetMode()
	STD_OFF: Mode WDGIF_SLOW_MODE is not supported. > If a pointer to a configuration structure which holds WDGIF_SLOW_MODE as Default Mode is passed to the API Wdg_30_Sbc_Init(), the development error WDG_E_PARAM_CONFIG is raised and the module stays uninitialized. > If the API Wdg_30_SetMode() is called with Mode parameter WDGIF_SLOW_MODE, the development error WDG_E_PARAM_MODE is raised. The function returns E_NOT_OK and the mode setup stays untouched.

Table 4-4 Macro WDG_30_SBC_SLOW_MODE_ALLOWED

Macro name	WDG_30_SBC_OFF_MODE_ALLOWED (STD_ON /STD_OFF)
Mapped from	SBC_30_<derivate>_WDG_OFF_MODE_ALLOWED (STD_ON/STD_OFF)
Settings:	STD_ON: Mode WDGIF_OFF_MODE is supported and can be used as Default Mode in the configuration structure passed to Wdg_30_Sbc_Init() or requested with Wdg_30_Sbc_SetMode()
	STD_OFF: Mode WDGIF_OFF_MODE is not supported. > If a pointer to a configuration structure which holds WDGIF_OFF_MODE as Default Mode is passed to the API Wdg_30_Sbc_Init(), the development error WDG_E_PARAM_CONFIG is raised and the module stays uninitialized. > If the API Wdg_30_SetMode() is called with Mode parameter WDGIF_OFF_MODE, the development error WDG_E_PARAM_MODE is raised. The function returns E_NOT_OK and the mode setup stays untouched.

Table 4-5 Macro WDG_30_SBC_OFF_MODE_ALLOWED

4.3 Trigger timing

4.3.1 Gpt Channel

The watchdog driver module uses a Gpt Channel for trigger timing. This Gpt Channel (Parameter `WdgTriggerTimer`) has to be configured in Channel Mode `GPT_CH_MODE_CONTINUOUS`.

The notification parameter (`GptNotification`) of the referenced channel has to be configured to the watchdog callback service `Wdg_30_Sbc_Cbk_GptNotificationTrigger()`.

4.3.2 Triggering time

The triggering time is adapted to the configuration of the referenced Sbc driver module.

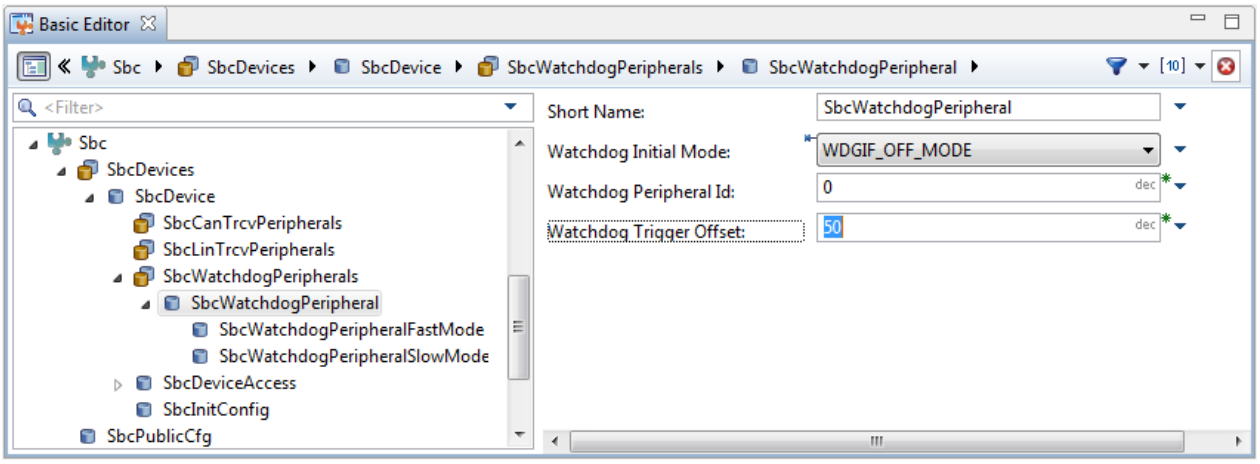


Figure 4-1 Configuration of SbcWatchdogPeripheral in DaVinci Configurator

The cyclic triggering time is calculated for slow and fast watchdog mode according the formula

`SbcWatchdogTriggerPeriod` [ms] × `SbcWatchdogTriggerOffset` [%].

The values are converted into ticks for the referenced Gpt channel and are stored in the array `TriggerTimerCounter` of the configuration structure Table 5-2

Note
The mentioned parameters of the Sbc module configuration have to be configured with respect to the requirements of the watchdog hardware of the SBC chip. (E.g. Open/Closed window, timeout).

The box below shows an example of a triggering time calculation.



Example

If the Sbc module is configured with the following values

- `SbcWatchdogTriggerPeriod`$_{Fast}$: 500 ms
- `SbcWatchdogTriggerPeriod`$_{Slow}$: 1000 ms
- `SbcWatchdogTriggerOffset:50`

the watchdog drivre module will trigger the watchdog cyclically every

- 250ms (500ms × 50%) in `WDGIF_FAST_MODE`
- 500ms (1000ms × 50%) in `WDGIF_SLOW_MODE`

5 API Description

For a detailed API description see [3].

5.1 Type Definitions

Types defined by the WDG are described in this chapter.

Type Name	C-Type	Description	Value Range
Wdg_ConfigType	struct	This is the configuration structure. A pointer to this structure has to be provided to the API `Wdg_Init`	

Table 5-1 Type Defenitions

Table 5-2 describes the members of `Wdg_ConfigType` which also is described

Wdg_ConfigType

Struct Element Name	C-Type	Description	Value Range
`EModeFailed`	`Dem_EventType`	This member is the DEM error corresponding to BSWMD parameter `WDG_E_MODE_FAILED`	uint16 (1…65535)
`EDisableRejected`	`Dem_EventType`	This member is the DEM error corresponding to BSWMD parameter `WDG_E_DISABLE_REJECTED`	uint16 (1…65535)
`DefaultMode`	`WdgIf_ModeType`	Mode which should be set during initialization.	`WDGIF_OFF_MODE`
			`WDGIF_SLOW_MODE`
			`WDGIF_FAST_MODE`
`*SbcSetModeFctPtr`	`Function pointer`	Pointer to the function of the Sbc module to set the watchdog mode	
`*SbcTriggerFctPtr`	`Function pointer`	Pointer to the function of the Sbc module to trigger the watchdog mode	
`TriggerTimer`	`Gpt_ChannelType`	GptChannel that is used for trigger timing	`Implementation specific`
`TriggerTimerFreq`	`uint32`	Frequency of referenced TriggerTimer	`Implementation specific`
`TriggerTimerCounter[2]`	`Gpt_ValueType`	Values of Gpt ticks for SLOW/FAST mode.	`Implementation specific`

Table 5-2 Wdg_ConfigType

5.2 Services used by WDG

Additional to the services described in [3] the module uses the following services

Component	API
Gpt	Gpt_StartTimer() Gpt_StopTimer() Gpt_EnableNotification()
Sbc	Sbc_Wdg_SetMode Sbc_Wdg_Trigger()

Table 5-3 Services used by the WDG

6 Glossary and Abbreviations

6.1 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 6-1 Abbreviations

7 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com